

Avaliação da Qualidade e Segurança de Pull Requests Gerados por Agentes

1st Vithor Barros Pileco

dept. de Informática

UFRGS

Porto Alegre, Brasil

00326674@ufrgs.br

2nd Laura Becker Ramos

dept. de Informática

UFRGS

Porto Alegre, Brasil

00326890@ufrgs.br

Abstract—A Inteligência Artificial Generativa está sendo rapidamente integrada aos fluxos de trabalho de desenvolvimento, automatizando desde a completude de código até a criação autônoma de Pull Requests (Agentic-PRs). Apesar do amplo uso, o impacto desses agentes na qualidade, manutenibilidade e segurança do código permanece uma questão em aberto. Este artigo utiliza o dataset do MSR 2026 Mining Challenge (AIDev) para conduzir um estudo empírico em larga escala sobre 1.590 Pull Requests em Python gerados por cinco agentes: Claude Code, GitHub Copilot, Cursor, Devin e OpenAI Codex. Através de ferramentas de análise estática (Flake8, Pylint e Bandit), mensuramos a variação líquida (Δ) em Linhas de Código (LOC), complexidade cognitiva e vulnerabilidades de segurança. Nossos resultados revelam uma dicotomia: enquanto agentes como Claude Code e Codex tendem a reduzir a complexidade cognitiva, frequentemente introduzem um volume maior de erros de linter e instabilidades sintáticas. Crucialmente, identificamos vulnerabilidades de alta severidade introduzidas pela IA, como uso inseguro de subprocessos e hashing fraco. Além disso, a análise do comportamento de aceitação (merge) expõe uma tendência preocupante: PRs que aumentam a complexidade cognitiva possuem uma alta taxa de aceitação, sugerindo que a revisão humana pode estar priorizando funcionalidade em detrimento da manutenibilidade a longo prazo. Conclui-se que a integração de agentes de IA exige barreiras automatizadas de CI/CD robustas para prevenir o acúmulo de dívida técnica.

Index Terms—Generative AI, Agentic Workflow, Code Quality, Software Security, Mining Software Repositories.

I. INTRODUÇÃO

A inteligência artificial (IA) generativa tem transformado profundamente o desenvolvimento de software. Modelos como GitHub Copilot, OpenAI Codex e ChatGPT vêm sendo amplamente adotados para acelerar a escrita de código, automatizar tarefas repetitivas e ampliar a produtividade das equipes. Diversas organizações têm relatado ganhos expressivos ao incorporar essas ferramentas ao ciclo de desenvolvimento: um estudo conduzido pela GitHub em parceria com a Accenture mostrou que desenvolvedores que utilizam o Copilot conseguem programar até 55% mais rápido, além de relatarem maior confiança na qualidade do código produzido (1). De modo semelhante, a HP reportou avanços significativos em eficiência após integrar o Copilot às suas equipes de engenharia (2).

Apesar desses benefícios, cresce a preocupação em relação aos riscos que acompanham o uso da IA como assistente

direto de programação. Estudos recentes apontam que uma parcela substancial do código gerado por modelos generativos apresenta vulnerabilidades de segurança ou práticas inadequadas de engenharia. Uma investigação conduzida pela Veracode identificou que cerca de 45% do código produzido por LLMs contém falhas exploráveis, incluindo problemas de validação, uso inseguro de bibliotecas e padrões suscetíveis a ataques (3). Pesquisas acadêmicas também apontam que vulnerabilidades podem surgir ou se intensificar em código gerado por IA. Aydin e Bahtiyar (4), por exemplo, mostram que LLMs frequentemente introduzem falhas de segurança recorrentes ao produzir código JavaScript.

Devido a esse problema, neste trabalho propomos investigar se as alterações realizadas por diferentes agentes resultam em melhorias mensuráveis nas principais métricas de qualidade de código, como número de linhas, complexidade cognitiva e ocorrência de vulnerabilidades em projetos de código em Python. Essa investigação será realizada diretamente sobre o conjunto de dados disponibilizado pelo Mining Challenge da MSR ¹. Essa base massiva oferece uma oportunidade para observar, em larga escala, como a IA tem modificado código real e quais padrões de qualidade e possíveis anti-patterns e falhas de segurança emergem dessas intervenções. Dessa forma, pretendemos contribuir para o debate sobre o uso responsável de ferramentas de IA no desenvolvimento, oferecendo indicações para a construção de práticas mais seguras, eficazes e sustentáveis no ciclo de engenharia de software.

II. TRABALHOS RELACIONADOS

A crescente adoção de modelos de linguagem no desenvolvimento de software tem motivado pesquisas empíricas baseadas em mineração de repositórios, com foco em compreender como a IA influencia a qualidade, segurança e evolução do código. Nesse cenário, Rabbi, Champa, Zibran e Islam (5) que mineraram o dataset DevGPT para analisar quase dois mil trechos de código gerados por ChatGPT. Os autores avaliam correção, complexidade e vulnerabilidades, mostrando que modelos de IA podem produzir soluções funcionais, mas

¹MSR 2026 Mining Challenge: <https://2026.msrfconf.org/track/msr-2026-mining-challenge>

com falhas recorrentes. Esse trabalho demonstra a relevância de análises sistemáticas baseadas em dados para compreender o comportamento da IA na prática.

Em uma linha de investigação similar, Jamil et al. (6) realizaram um estudo empírico comparativo entre códigos gerados por LLMs (GPT-3.5 e GPT-4) e soluções humanas do dataset *HumanEval*. Este trabalho é particularmente relevante para nossa proposta por validar metodologicamente o uso de ferramentas de análise estática para comparação de qualidade: os autores utilizaram *Radon* e *Complexity* para mensurar complexidade cognitiva, além de *Pylint* e *Bandit* para corretude e segurança. Os resultados indicaram que, sob estratégias de *prompting* aprimoradas, o GPT-4 foi capaz de produzir código com métricas de manutenibilidade e segurança superiores às soluções humanas de referência. No entanto, enquanto Jamil et al. focam em *snippets* de código isolados em um ambiente controlado de benchmark, nosso trabalho busca expandir essa análise para o ecossistema de *Agentic-PRs*, onde a qualidade do código interage diretamente com a complexidade de projetos de software reais e legados.

Outro estudo diretamente relacionado é “*What Developers Ask to ChatGPT in GitHub Pull Requests?*” (7), que investiga o uso da IA dentro de pull requests reais do GitHub. Por meio de mineração em repositórios públicos, os autores identificam padrões de interação entre desenvolvedores e ChatGPT, como pedidos de refatoração, explicação de código e correção de erros. Embora o estudo não avalie métricas quantitativas de qualidade, ele revela como a IA já está integrada ao fluxo colaborativo, reforçando a importância de investigar seus impactos técnicos, o que motiva a proposta deste trabalho.

Além das pesquisas centradas em IA, há uma base consolidada de estudos em mineração de código em larga escala que investigam a relação entre práticas de desenvolvimento, qualidade e defeitos. Um exemplo representativo é o trabalho de Ray, Posnett, Filkov e Devanbu (8), que realizou um estudo massivo em centenas de repositórios do GitHub para analisar como diferentes linguagens de programação se relacionam com métricas como complexidade, LOC e propensão a defeitos. Ao correlacionar características estruturais do código com a ocorrência de problemas reais, o estudo demonstra como análises quantitativas baseadas em repositórios extensos podem revelar padrões consistentes de qualidade, abordagem metodológica diretamente alinhada ao objetivo do presente trabalho.

Em conjunto, esses estudos demonstram que a mineração de dados aplicada a código gerado por IA é uma abordagem essencial para compreender os riscos, limitações e padrões de comportamento desses modelos em ambientes reais. Diferentemente de análises isoladas ou experimentais, abordagens baseadas em repositórios massivos permitem observar como a IA se manifesta no fluxo real de desenvolvimento. Nesse contexto, o presente trabalho pretende analisar, em larga escala, como agentes de IA influenciam métricas objetivas de qualidade em pull requests, utilizando a base do MSR 2026 Mining Challenge como fonte principal de evidências.

III. METODOLOGIA

A. Dataset

Para conduzir este estudo, utilizamos o dataset *AIDev*, introduzido por Li et al. (9), que constitui a primeira base de dados em larga escala focada na captura sistemática de *Agentic-PRs*, pull requests gerados de forma autônoma ou semi-autônoma por agentes de inteligência artificial.

O dataset foi construído através da mineração de repositórios no GitHub, utilizando a API REST e consultas direcionadas para identificar assinaturas digitais de cinco dos principais agentes de codificação do mercado: OpenAI Codex, Devin, GitHub Copilot, Cursor e Claude Code. A identificação dos PRs baseou-se em padrões específicos, como nomes de bots (`devin-ai-integration[bot]`), prefixos de branches (`head:codex/`) e mensagens explícitas no corpo do PR (“Co-Authored-By: Claude”).

O *AIDev* abrange um total de 932.791 pull requests, originados de 116.211 repositórios e envolvendo 72.189 desenvolvedores distintos. A coleta de dados cobre o período até 01 de agosto de 2025. O dataset inclui metadados ricos que permitem a reconstrução do ciclo de vida das contribuições.

B. Métricas de Qualidade de Software

Para quantificar o impacto dos agentes de IA na manutenibilidade e na estrutura do código, selecionamos três métricas estáticas comuns para avaliação de qualidade de código. Para obtê-las, calculamos as métricas de qualidade para o estado do repositório imediatamente anterior (S_{pre}) e posterior (S_{post}) às mudanças introduzidas pelos Pull Request. A métrica atribuída ao agente é definida pela variação líquida (Δ) observada entre esses estados:

$$\Delta M = \sum_{f \in R} M(f, S_{post}) - \sum_{f \in R} M(f, S_{pre}) \quad (1)$$

Onde R representa o conjunto de arquivos do repositório e M a função de medição (e.g., complexidade ou LOC). Essa abordagem garante que refatorações (que reduzem complexidade) ou deleções de código sejam contabilizadas corretamente como impactos negativos ou positivos, respectivamente.

1) *Lines of Code (LOC)*: A contagem de linhas de código (LOC) é uma métrica de tamanho tradicional que, embora simples, serve como um indicador primário de verbosidade e esforço de manutenção. Consideramos apenas as linhas lógicas de código (SLOC).

2) *Complexidade Cognitiva (Cognitive Complexity)*: A métrica de Complexidade Cognitiva, formalizada pela SonarSource, que propõe abandonar modelos puramente matemáticos em favor de uma abordagem baseada na intuição do programador para mensurar a *inteligibilidade* (understandability) do fluxo de controle. Um alto valor de complexidade cognitiva de uma função indica um alto custo cognitivo para entendê-la e mantê-la.

C. Ferramentas de Análise Estática

Para garantir a reprodutibilidade e a escalabilidade da avaliação em milhares de pull requests, selecionamos três ferramentas de análise estática amplamente adotadas no ecossistema Python. A escolha dessas ferramentas baseia-se na capacidade de execução via linha de comando (CLI) e na geração de saídas estruturadas (JSON), essenciais para o nosso pipeline de mineração.

1) *Flake8*: O *Flake8* é uma ferramenta modular que agrega verificações de estilo (PEP 8), erros de sintaxe e complexidade. Neste estudo, utilizamos o *Flake8* não apenas para verificação de estilo, mas principalmente como o motor de cálculo da métrica de complexidade cognitiva.

2) *Bandit*: Focado exclusivamente em segurança, o *Bandit* processa o código fonte para construir uma AST e varrê-la em busca de padrões conhecidos de vulnerabilidade. Diferente de ferramentas dinâmicas, o *Bandit* não executa o código, o que o torna seguro para analisar sugestões de IA não confiáveis. Ele categoriza falhas em níveis de severidade (Baixa, Média, Alta) que depois são utilizadas para calcular o delta e obter possíveis falhas de segurança introduzidas ou removidas pelos agentes.

3) *Pylint*: O *Pylint* atua como um linter de propósito geral, capaz de detectar uma ampla gama de defeitos, desde erros de sintaxe fatais até *code smells* e antipadrões de programação. Sua análise profunda do fluxo de dados permite identificar erros lógicos que passariam despercebidos em uma análise puramente sintática. Para este estudo, contabilizamos a variação líquida de mensagens de erro e avisos (*warnings*) para determinar se o código produzido pelos agentes degrada a qualidade interna do projeto em comparação à base existente.

D. Arquitetura da Implementação

Para viabilizar a análise em larga escala, desenvolvemos um *pipeline* de automação modular em Python². A solução foi arquitetada para permitir extensibilidade para outras linguagens, dividindo-se em dois componentes fundamentais:

- **Orquestração e Dados:** Um módulo central coordena a ingestão dos metadados do dataset *AIDev*, gerencia o ciclo de vida da análise de cada repositório, como os comandos necessários no git, e persiste os resultados calculados.
- **Motor de Análise Polimórfico:** A extração de métricas segue uma arquitetura baseada em classes abstratas. Implementações concretas (e.g., *PythonAnalyzer*) estendem a base comum para integrar ferramentas específicas (*Flake8*, *Bandit*, *Pylint*), processando as Árvores de Sintaxe Abstrata (AST) e saídas de linters para computar o Δ de qualidade e segurança.

A estrutura permite que nossa análise seja estendida para outras linguagens, bastando a implementação de um analisador específico para ela.

²O código fonte e os scripts de reprodução estão disponíveis em: <https://github.com/Vithor112/msr2026>

E. Análise quantitativa

Com as métricas coletadas pelo pipeline, realizamos uma análise quantitativa comparando o estado do código antes e depois das mudanças introduzidas pelos agentes. Essa etapa permite medir a variação em LOC, complexidade e segurança, identificando tendências gerais no impacto da IA sobre a qualidade do software. A partir desses dados, examinamos padrões agregados e diferenças entre agentes, oferecendo uma visão objetiva do efeito dessas contribuições em larga escala.

TABLE I
IMPACTO MÉDIO POR PULL REQUEST DOS AGENTES DE IA NAS
MÉTRICAS DE QUALIDADE

Agente	Vuln. Segurança (Alta)	Erros Gerais	Alta Complex. Cognitiva
Claude Code	0.0271	6.2331	-13.7331
Copilot	0.0150	0.3684	0.1729
Cursor	-0.0455	0.9773	0.8409
Devin	0.0000	0.2683	0.5122
OpenAI Codex	-0.0379	0.0536	-6.9432

Nota: Valores negativos indicam redução média da métrica (melhoria), enquanto valores positivos indicam aumento (piora).

IV. RESULTADOS

Foram analisados mais de 1590 Pull Request de 330 repositórios de python distintos. Dos 1590 PRs, 738 foram do Claude, 634 do Codex, 133 do Copilot, 44 Cursor e 41 do Devin. A Tabela I apresenta a média de diferentes métricas, cujo valores negativos indicam uma melhora e os valores positivos indicam uma piora depois das mudanças dos agentes. A coluna Vuln. Segurança (Alta) mostra o número de erros de segurança de alta periculosidade adicionados ou removidos. Já as colunas de Alta Complex. Cognitiva indica o número de funções introduzidas ou removidas com valores maiores que 7.

Na Tabela II e na Tabela III categorizamos os erros sintáticos e lógicos (detectados pelo Pylint) e as vulnerabilidades de segurança (detectadas pelo Bandit) mais recorrentes no código gerado pelos agentes.

Na Figura 1 é mostrado a influência das métricas de qualidade utilizadas na aceitação ou rejeição das mudanças dos agentes.

TABLE II
TOP 10 ERROS DE LINTER (PYLINT) EM CÓDIGO GERADO POR IA

Tipo de Erro (Pylint)	Contagem
error_no-self-argument	2.063
error_used-before-assignment	1.863
error_invalid-hash-returned	1.416
error_no-value-for-parameter	413
error_syntax-error	219
error_unexpected-keyword-arg	187
error_assignment-from-no-return	111
error_unsubscriptable-object	88
error_abstract-class-instantiated	53
error_too-many-function-args	53

TABLE III
VULNERABILIDADES DE ALTA SEVERIDADE (BANDIT)

Vulnerabilidade (Bandit)	Contagem
hashlib_HIGH	84
subprocess_popen_with_shell_equals_true_HIGH	44
jinja2_autoescape_false_HIGH	6
flask_debug_true_HIGH	1

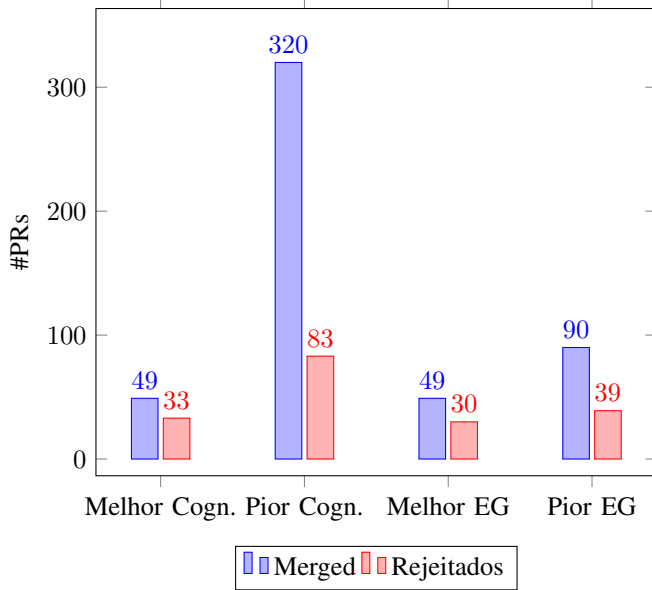


Fig. 1. Cogn: Complexidade Cognitiva; EG: Erros Gerais Linter

V. CONCLUSÕES

Os dados indicam que, embora certos agentes (como Claude Code e OpenAI Codex) demonstrem capacidade de reduzir a complexidade cognitiva do código legado (Tabela I), essa melhoria frequentemente vem acompanhada de um aumento substancial em erros de *linter* e instabilidade sintática, evidenciado pela alta taxa de erros gerais no Claude Code (6,23 por PR) e pela recorrência de falhas básicas como *error_no_self_argument* (Tabela II).

Além disso, a detecção de vulnerabilidades de alta severidade, como uso inseguro de `subprocess` e `hashing` fraco (Tabela III), reforça que a geração de código por IA ainda não garante segurança *by-design*. Um achado crítico deste trabalho reside na análise de aceitação (Fig. 1): observou-se uma tendência preocupante onde PRs que pioram a complexidade cognitiva possuem uma alta taxa de *merge* (320 aceitos contra 83 rejeitados). Isso sugere que a facilidade de geração de código pode estar levando à aceitação de soluções funcionais, porém de difícil manutenção, sem o devido escrutínio humano.

Conclui-se, portanto, que a integração de agentes de IA exige pipelines robustos de análise estática (CI/CD) para mitigar a introdução de dívida técnica e vulnerabilidades. Trabalhos futuros devem investigar a correlação entre a aceitação de código complexo gerado por IA e a incidência de *bugs* em produção a longo prazo.

REFERÊNCIAS

- [1] Y. Gao and G. C. Research, “Research: Quantifying github copilot’s impact in the enterprise with accenture,” <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/>, May 13 2024, acessado em: 17 nov. 2025.
- [2] Microsoft News Center Brasil, “Hp adota github copilot e aumenta a inovação e a velocidade do software,” <https://news.microsoft.com/pt-br/hp-adota-github-copilot-e-aumenta-a-inovacao-e-a-velocidade-do-software/>, Jul 2024, acessado em: 17 nov. 2025.
- [3] TechRadar Pro, “Nearly half of all code generated by ai found to contain security flaws – even big llms affected,” <https://www.techradar.com/pro/nearly-half-of-all-code-generated-by-ai-found-to-contain-security-flaws>, Aug 2025, acessado em: 17 nov. 2025.
- [4] D. Aydın and Ş. Bahtiyar, “Security vulnerabilities in ai-generated javascript: A comparative study of large language models,” in *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 2025, pp. 200–205.
- [5] M. F. Rabbi, A. I. Champa, M. F. Zibran, and M. R. Islam, “Ai writes, we analyze: The chatgpt python code saga,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 177–181.
- [6] M. T. Jamil, S. Abid, and S. Shamail, “Can llms generate higher quality code than humans? an empirical study,” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025, pp. 478–489.
- [7] J. R. Silva, C. E. C. Dantas, and M. A. Maia, “What developers ask to chatgpt in github pull requests? an exploratory study,” *arXiv preprint arXiv:2508.17161*, 2025.
- [8] B. Ray, D. Posnett, V. Filkov, and P. Devanbu, “A large-scale study of programming languages and code quality in github,” in *Proceedings of the 2014 International Symposium on Foundations of Software Engineering (FSE 2014)*, Hong Kong, China, 2014, pp. 155–165.
- [9] H. Li, H. Zhang, and A. E. Hassan, “The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering,” 2025, arXiv preprint arXiv:2507.15003. [Online]. Available: <https://arxiv.org/abs/2507.15003>